HW07

Sam Ly

March 24, 2026

Chapter 13

https://github.com/samlyme/Assignments/tree/cs4080/ch13/challenge

1. Lox supports only single inheritance—a class may have a single superclass and that's the only way to reuse methods across classes. Other languages have explored a variety of ways to more freely reuse and share capabilities across classes: mixins, traits, multiple inheritance, virtual inheritance, extension methods, etc.

   If you were to add some feature along these lines to Lox, which would you pick and why? If you're feeling courageous (and you should be at this point), go ahead and add it.

   I originally set out to implement a trait system, but they don't really "work" in dynamically typed languages. Instead, I implemented a way for programmers to extend the funcitonality of a *class* after it has been declared. This is similar to traits, but a bit more loosey goosey, without the strict typing system.

   All future and *current* instances of the class will receive the new methods. Because everything is bound dynamically anyway, this actually wasn't too difficult to implement.

   Example of the syntax: (the 'C' class is defined earlier)

   ```
   var sam = C("sam");
   var bob = C("bob");

   // error
   // sam.added();

   // Cursed syntax for introspection
   class C + {
       added() {
           print this.name + "this method was added after C was defined";
       }
   }

   sam.added();
   bob.added();
   // from: sam
   // this method was added after C was defined
   // from: bob
   // this method was added after C was defined
   ```

   The main difficulties of implementing this is in the Resolver. Because there is no way to statically check if the baseClass of this statement is a subclass or not. I ended up comprimising, letting the super always have access to super, and defering the error to runtime.

```
@Override
public Void visitClassPlusStmt(Stmt.ClassPlus stmt) {
    ClassType enclosingClass = currentClass;

    currentClass = ClassType.SUBCLASS; // although not "correct", good enough.
    // No good way to statically determine if this class is a subclass, so we just
    // let the user use SUPER.

    resolve(stmt.baseClass);
    beginScope();
    scopes.peek().put("this", true);

    for (Stmt.Function method : stmt.methods) {
        FunctionType declaration = FunctionType.METHOD;
        if (method.name.lexeme.equals("init")) {
            declaration = FunctionType.INITIALIZER;
        }
        resolveFunction(method, declaration);
    }
    endScope();

    currentClass = enclosingClass;

    return null;
}
```

2. Take out Lox's current overriding and super behavior and replace it with BETA's semantics. In short:

   When calling a method on a class, prefer the method highest on the class's inheritance chain.

   Inside the body of a method, a call to inner looks for a method with the same name in the nearest subclass along the inheritance chain between the class containing the inner and the class of this. If there is no matching method, the inner call does nothing.

   I did this by replacing the 'findMethod' method inside of the LoxClass class with 'findMethodRoot' that recursively returns the superclass's method, with the closure modified to have an 'inner' variable bound.

   Most other code was just plumbing to make sure the variables end up where they're supposed to.

```
LoxFunction findMethodRoot(String name) {
    if (superclass == null) {
        if (methods.containsKey(name)) return methods.get(name);
        return null;
    }

    if (methods.containsKey(name)) {
        return superclass.findMethodRoot(name).bindInner(methods.get(name));
    }
    return superclass.findMethodRoot(name);
}
```

3. In the chapter where I introduced Lox, I challenged you to come up with a couple of features you think the language is missing. Now that you know how to build an interpreter, implement one of those features.

I made a mistake by wanting to have type checking since it is an extremely difficult feature to implement. Nevertheless, I have given it my best shot, despite the absolute cursed code that was required.

This feature required such extensive changes, that I have separated from the other challenge branches. You can find the code here:

https://github.com/samlyme/Assignments/tree/cs4080/ch13/challenge-3

The type checking is extremely limited. It only tracks the class of variables who were intialized with a constructor of a class. All other types are ignored. Also, the type checking doesn't fully function, since reassigning a variable breaks the invariant held in the Resolver class. There is no way to know what the right hand side of an set expression evaluates to, and this type of static analysis will likely require a total refactor of the Resolver class, or an entirely new TypeChecker class.

To track what methods a class has access to, we add some code to the visitClassStmt code:

```
        scopes.peek().put("this", true);

        Set<String> methodNames = new HashSet<>();
        classProperties.put(stmt.name.lexeme, methodNames);

        for (Stmt.Function method : stmt.methods) {
            FunctionType declaration = FunctionType.METHOD;
            methodNames.add(method.name.lexeme);
            System.out.println("Add to class " + stmt.name.lexeme + " property: " + method.name.lex

            if (method.name.lexeme.equals("init")) {
                declaration = FunctionType.INITIALIZER;
            }
            resolveFunction(method, declaration);
        }
```

Then, we go into the initializer of the class to track the available fields:

```
    private void resolveFunction(Stmt.Function function, FunctionType type) {
        FunctionType enclosingFunction = currentFunction;
        currentFunction = type;

        if (currentFunction == FunctionType.INITIALIZER && currentClassName != null) {
            System.out.println("looking for this. = <whatever> in constructor.");
            for (Stmt stmt : function.body) {
                // cursed string parsing
                if (stmt instanceof Stmt.Expression) {
                    if (((Stmt.Expression) stmt).expression instanceof Expr.Set) {
                        Expr object = ((Expr.Set) ((Stmt.Expression) stmt).expression).object;
                        Token name = ((Expr.Set) ((Stmt.Expression) stmt).expression).name;
                        if (object instanceof  Expr.This) {
                            classProperties.get(currentClassName).add(name.lexeme);
                        }
//                          System.out.println("object: " + object + " name: " + name);

                    }
                }
            }
        }
```

```
        beginScope();
        for (Token param : function.params) {
            declare(param);
            define(param);
        }
        resolve(function.body);
        endScope();

        currentFunction = enclosingFunction;
    }
```

The only way I was able to implement the basic type checking feature was because I explicilty checked for constructor calls.

```
    @Override
    public Void visitVarStmt(Stmt.Var stmt) {
        declare(stmt.name);
        if (stmt.initializer != null) {
            resolve(stmt.initializer);
        }

        System.out.println("Declare var");
        if (stmt.initializer instanceof Expr.Call) {
            System.out.println("Initializer is of type Expr.Call");
            Expr callee = ((Expr.Call) stmt.initializer).callee;
            System.out.println("Initializer callee" + callee);
            if (callee instanceof  Expr.Variable) {
                System.out.println("Initializer callee is of type Variable");
                String lexeme = ((Expr.Variable) callee).name.lexeme;
                if (classProperties.containsKey(lexeme)) {
                    System.out.println("Initializer callee is a constructor");
                    // we have found a constructor
                    varClass.put(stmt.name.lexeme, lexeme);
                    System.out.println(stmt.name.lexeme + " is known to be of class " + varClass.ge
                }
            }
        }
        define(stmt.name);
        return null;
    }
```

PS. This was genuinely one of the most difficult problems to implement in Java. The lack of pattern matching made it very verbose to check for specific expressions whose type we know at compile time, hence the massive number of 'instanceof' calls and casting.

Chapter 14

https://github.com/samlyme/Assignments/tree/cs4080/ch14/challenge

1. Our encoding of line information is hilariously wasteful of memory. Given that a series of instructions often correspond to the same source line, a natural solution is something akin to run-length encoding of the line numbers.

Devise an encoding that compresses the line information for a series of instructions on the same line. Change writeChunk() to write this compressed form, and implement a getLine() function that, given the index of an instruction, determines the line where the instruction occurs.

We do this by making the 'lines' field of the Chunk struct be its own dynamic array, independent of the 'code' dynamic array. This way, when we want to add a new line, we can either increment a count, or add a new entry to lines, without having the sync the size with the 'code'.

```
void writeChunk(Chunk* chunk, uint8_t byte, int line) {
    if (chunk->capacity <= chunk->count) {
        int oldCapacity = chunk->capacity;

        chunk->capacity = GROW_CAPACITY(oldCapacity);

        chunk->code = GROW_ARRAY(uint8_t, chunk->code, oldCapacity, chunk->capacity);
        // chunk->lines = GROW_ARRAY(int, chunk->lines, oldCapacity, chunk->capacity);
    }
    chunk->code[chunk->count] = byte;
    chunk->count++;

    if (chunk->lineCapacity <= chunk->lineCount) {
        int oldCapacity = chunk->lineCapacity;

        chunk->lineCapacity = GROW_CAPACITY(oldCapacity);
        chunk->lines = GROW_ARRAY(int, chunk->lines, oldCapacity, chunk->lineCapacity);
        // printf("grow lines\n");
    }

    // assume that lines is homotopic increasing.
    if (chunk->lineCount == 0) {
        chunk->lines[0] = 1;
        chunk->lines[1] = line;
        chunk->lineCount = 2;
        // printf("init lines\n");
    } else if (line == chunk->lines[chunk->lineCount - 1])  {
        chunk->lines[chunk->lineCount - 2]++;
        // printf("incr lines\n");
    } else {
        chunk->lines[chunk->lineCount] = 1;
        chunk->lines[chunk->lineCount + 1] = line;
        chunk->lineCount += 2;
        printf("new  lines\n");
    }
}
```

Now, we just do a linear search for 'getLine'.

```
int getLine(Chunk* chunk, int offset) {
    int codeIdx = 0;
    int lineIdx = 0;
    while (codeIdx < offset) {
        codeIdx++;
        if (chunk->lines[lineIdx] == 0) {
            lineIdx += 2;
```

```
        } else {
            chunk->lines[lineIdx]--;
        }
    }

    return chunk->lines[lineIdx + 1];
}
```

2. Defining two instructions seems to be the best of both worlds. What sacrifices, if any, does it force on us?

   The implementation is relatively simple. We just check if the current number of constants is less than 256. If this is the case, a normal `OP_CONSTANT` is written, otherwise, we issue `OP_CONSTANT_LONG`.

```
void writeConstant(Chunk* chunk, Value value, int line) {
    int constantIdx = addConstant(chunk, value);

    if (constantIdx < 256) {
        writeChunk(chunk, OP_CONSTANT, line);
        writeChunk(chunk, constantIdx, line);
    } else {
        writeChunk(chunk, OP_CONSTANT_LONG, line);
        uint8_t a = constantIdx >> 16;
        uint8_t b = constantIdx >> 8;
        uint8_t c = constantIdx;
        writeChunk(chunk, a, line);
        writeChunk(chunk, b, line);
        writeChunk(chunk, c, line);
    }
}
```

   The most obvious sacrifice is that it uses an extra 'instruction slot'. We only have 256 possible instructions because we encode them as a byte. This also adds a layer of complexity.

3. Our reallocate() function relies on the C standard library for dynamic memory allocation and freeing. malloc() and free() aren't magic. Find a couple of open source implementations of them and explain how they work. How do they keep track of which bytes are allocated and which are free? What is required to allocate a block of memory? Free it? How do they make that efficient? What do they do about fragmentation?

   At a very high level, malloc and free use doubly linked lists to track memory. When you request memory, malloc traverses the list and finds a block with enough memory, which it returns to you. Calling free takes your pointer and connects it to the previous block of free memory.